Experiment No. : 3
Title: To scrape data from a dynamic website

Batch: B3 **Roll No.: 16014225801** **Experiment No.: 2**

Aim: To develop a Python program using **Selenium WebDriver** to scrape data from a dynamic website by automating a web browser, locating webpage elements, interacting with them, and extracting the required information.

Resources Required

- Python 3.x
- Selenium
- VS Code / PyCharm / Jupyter Notebook
- Web browser – Chrome/Edge/Firefox

Target Website: Mention here the URL of your selected target website. Selected website should allow good amount of data to scrap and explore

---

Theory:

1. Static vs Dynamic Websites

A static webpage generally contains the required information in the HTML returned by the server. A dynamic webpage may load or modify content using:

- JavaScript
- AJAX
- API requests
- User interactions
- Asynchronous loading
-

Therefore, the HTML received initially may not contain all the information visible in the browser.

2. Static Website

Python → HTTP Request → Server → HTML → BeautifulSoup → Data

3. Dynamic Website

Python → Selenium → Browser → Website → JavaScript executes → Dynamic content appears → Selenium → Data

4. Why Selenium?

BeautifulSoup is primarily an HTML parser. It does not itself execute JavaScript. Selenium controls a real browser and allows a Python program to:

- Open webpages
- Click buttons
- Enter text
- Scroll
- Navigate pages
- Locate elements
- Extract rendered content

5. **Selenium WebDriver**

Selenium WebDriver provides programmatic control over a web browser.

Basic structure

```
from selenium import webdriver
driver = webdriver.Chrome()
driver.get("URL")
# Perform operations
driver.quit()
```

6. Selenium Locators

Selenium provides several ways to locate elements.

Locator	Purpose
By.ID	Locate using element ID
By.NAME	Locate using name attribute
By.CLASS_NAME	Locate using class
By.TAG_NAME	Locate using HTML tag
By.LINK_TEXT	Locate using complete link text
By.PARTIAL_LINK_TEXT	Locate using part of link text
By.CSS_SELECTOR	Locate using CSS Selector
By.XPATH	Locate using XPath

7. find_element() and find_elements()

Find one element:

```
element = driver.find_element(
    By.ID,
    "username"
)
```

Find multiple elements:

```
elements = driver.find_elements(
    By.TAG_NAME,
    "p"
)
```

Method	Purpose	Result
find_element()	Find one matching element	One WebElement
find_elements()	Find multiple matching elements	List of WebElements

8. Extracting Data

After locating an element, Selenium can extract information from it.

Requirement	Method
Visible text	element.text
HTML attribute	element.get_attribute()
Page title	driver.title
Current URL	driver.current_url

Example

```
price = driver.find_element(
    By.CSS_SELECTOR,
    ".price_color"
)
print(price.text)
```

9. Interacting with Elements

Selenium can also interact with webpage elements.

Method	Purpose
click()	Click an element
send_keys()	Enter text / keyboard input
clear()	Clear an input field

Example

```
search_box.send_keys("Python")
button.click()
```

10. Procedure

Students should perform the following activities.

Step 1 – Select a Dynamic Website

Select an appropriate dynamic website such as:

- An e-commerce product/review page
- A dynamic information dashboard
- A publicly accessible practice website

Do not use a website that prohibits automated access.

Step 2 – Open the Website Using Selenium

from selenium import webdriver
driver = webdriver.Chrome()
driver.get("WEBSITE_URL")

Step 3 – Inspect the Webpage

Use browser Developer Tools to identify:

- IDs
- Classes
- Tags
- Attributes
- CSS Selectors
- XPath

Step 4 – Locate Elements

Use appropriate Selenium locators.

Example:

```
from selenium.webdriver.common.by import By

element = driver.find_element(
   By.CSS_SELECTOR,
   "CSS_SELECTOR")
```

Step 5 – Extract Data

Use:

element.text or element.get_attribute("href")

Step 6 – Interact with Dynamic Elements

Where appropriate:

element.click() or element.send_keys("search term")

Step 7 – Extract Multiple Records

```
elements = driver.find_elements(
   By.CSS_SELECTOR,
   "CSS_SELECTOR"
    )
for element in elements:
   print(element.text)
```

Step 8 – Store the Extracted Data

Students should organize the extracted information into a suitable Python structure such as:

```
data = []
for element in elements:
   data.append(element.text)
```

Optionally, students can store the final data in CSV format.

Step 9 – Close the Browser

```
driver.quit()
```

11. Suggested Practical Task

Dynamic Product Information Scraper

Students can develop a scraper that extracts information such as:

Field	Example
Product Name	Laptop
Price	₹55,000
Rating	4.5
Review Count	125
Availability	In Stock
Product URL	URL

Workflow

Dynamic Website → Selenium WebDriver → Open Browser → Load Webpage → Inspect DOM → Locate Elements → CSS Selector / XPath → Extract Data → Store Data

14. Complete Basic Program Structure

```
from selenium import webdriver
from selenium.webdriver.common.by import By

# Start browser
driver = webdriver.Chrome()

# Open website
driver.get("WEBSITE_URL")

# Locate elements
elements = driver.find_elements(
    By.CSS_SELECTOR,
    "CSS_SELECTOR"
)

# Extract data
for element in elements:
    print(element.text)

# Close browser
driver.quit()
```

Expected Output

The output depends on the selected website.
For example:
Product 1
₹45,000
4.5
In Stock

Product 2
₹62,000
4.3
In Stock

15. Activities to be Performed in the Lab

Students must complete the following:

No.	Activity
1	Install/import Selenium
2	Launch browser using Selenium

No.	Activity
3	Open selected dynamic website
4	Inspect webpage DOM
5	Identify suitable locators
6	Locate at least one element using ID/Class/Tag
7	Locate an element using CSS Selector
8	Locate an element using XPath
9	Extract visible text
10	Extract at least one attribute
11	Use find_elements() for multiple records
12	Perform at least one interaction using click() or send_keys()
13	Extract multiple records
14	Display the scraped data
15	Close the browser using driver.quit()

Post-Lab Questions:

AI Use-Case Integration

Climate Intelligence for Heatwave Monitoring, Prediction, and Early Warning

Students can conceptually consider a dynamic weather dashboard that displays:

- Temperature
- Humidity
- Heat index
- Weather alerts
- District information

Workflow

Dynamic Weather Dashboard → Selenium → Extract Weather Data → Clean / Structure Data → AI Mode → Heatwave Prediction → Early Warning

---

Code

Python:

```python
from __future__ import annotations
import argparse
import re
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Optional

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import Select, WebDriverWait


@dataclass
class WeatherData:
    district: str
    temperature_c: float
```

```
    humidity_percent: float
    heat_index_c: float
    weather_alert: str


def number_from_text(text: str) -> float:
    """Convert values such as '46 °C' and '38 %' into numbers."""
    match = re.search(r"-?\d+(?:\.\d+)?", text)
    if not match:
        raise ValueError(f"No number found in: {text!r}")
    return float(match.group())


def scrape_weather(driver: webdriver.Chrome, district: Optional[str]) -> WeatherData:
    """Interact with the dashboard, wait for its JavaScript data, then extract it."""
    wait = WebDriverWait(driver, 10)

    # Optional interaction: choose the requested district before refreshing.
    if district:
        Select(wait.until(EC.element_to_be_clickable((By.ID, "district-
select")))).select_by_visible_text(district)

    wait.until(EC.element_to_be_clickable((By.ID, "refresh-button"))).click()
    # Wait for JavaScript to fill a value, not only for the parent container
    # to become visible.
    wait.until(
        lambda browser: browser.find_element(By.ID, "temperature")
        .get_attribute("textContent")
        .strip()
    )

    # Update these locators if a real dashboard uses different HTML attributes.
    def value(selector: str) -> str:
        element = driver.find_element(By.ID, selector)
        return (element.text or element.get_attribute("textContent") or "").strip()

    district_text = value("district")
    temperature_text = value("temperature")
    humidity_text = value("humidity")
    heat_index_text = value("heat-index")
    alert_text = value("weather-alert")

    return WeatherData(
        district=district_text.strip(),
        temperature_c=number_from_text(temperature_text),
        humidity_percent=number_from_text(humidity_text),
        heat_index_c=number_from_text(heat_index_text),
        weather_alert=alert_text.strip(),
    )


def predict_heatwave(data: WeatherData) -> tuple[str, str]:
    """A transparent rule-based AI model for an easy-to-understand prototype."""
    if data.temperature_c >= 45 or data.heat_index_c >= 54:
        return "EXTREME", "Issue an immediate public heatwave warning."
    if data.temperature_c >= 42 or data.heat_index_c >= 46:
        return "HIGH", "Prepare cooling centres and advise people to limit outdoor activity."
    if data.temperature_c >= 38 or data.heat_index_c >= 41:
        return "MODERATE", "Monitor vulnerable groups and share heat-safety guidance."
    return "LOW", "Continue routine monitoring."


def main() -> None:
    parser = argparse.ArgumentParser(description="Selenium heatwave dashboard scraper")
    parser.add_argument("--district", default="Delhi", help="District to select in the demo
dashboard")
    parser.add_argument("--url", help="URL of a compatible weather dashboard")
    parser.add_argument("--headless", action="store_true", help="Run Chrome without opening a
window")
```

```
    args = parser.parse_args()

    dashboard_url = args.url or
Path(__file__).with_name("weather_dashboard.html").resolve().as_uri()
    options = webdriver.ChromeOptions()
    if args.headless:
        options.add_argument("--headless=new")

    # Selenium Manager obtains a suitable ChromeDriver automatically in Selenium 4.6+.
    with webdriver.Chrome(options=options) as driver:
        driver.get(dashboard_url)
        weather = scrape_weather(driver, args.district if not args.url else None)

    risk, action = predict_heatwave(weather)
    print("\nEXTRACTED WEATHER DATA")
    for key, value in asdict(weather).items():
        print(f"{key}: {value}")
    print("\nHEATWAVE PREDICTION")
    print(f"Risk level: {risk}")
    print(f"Early warning: {action}")

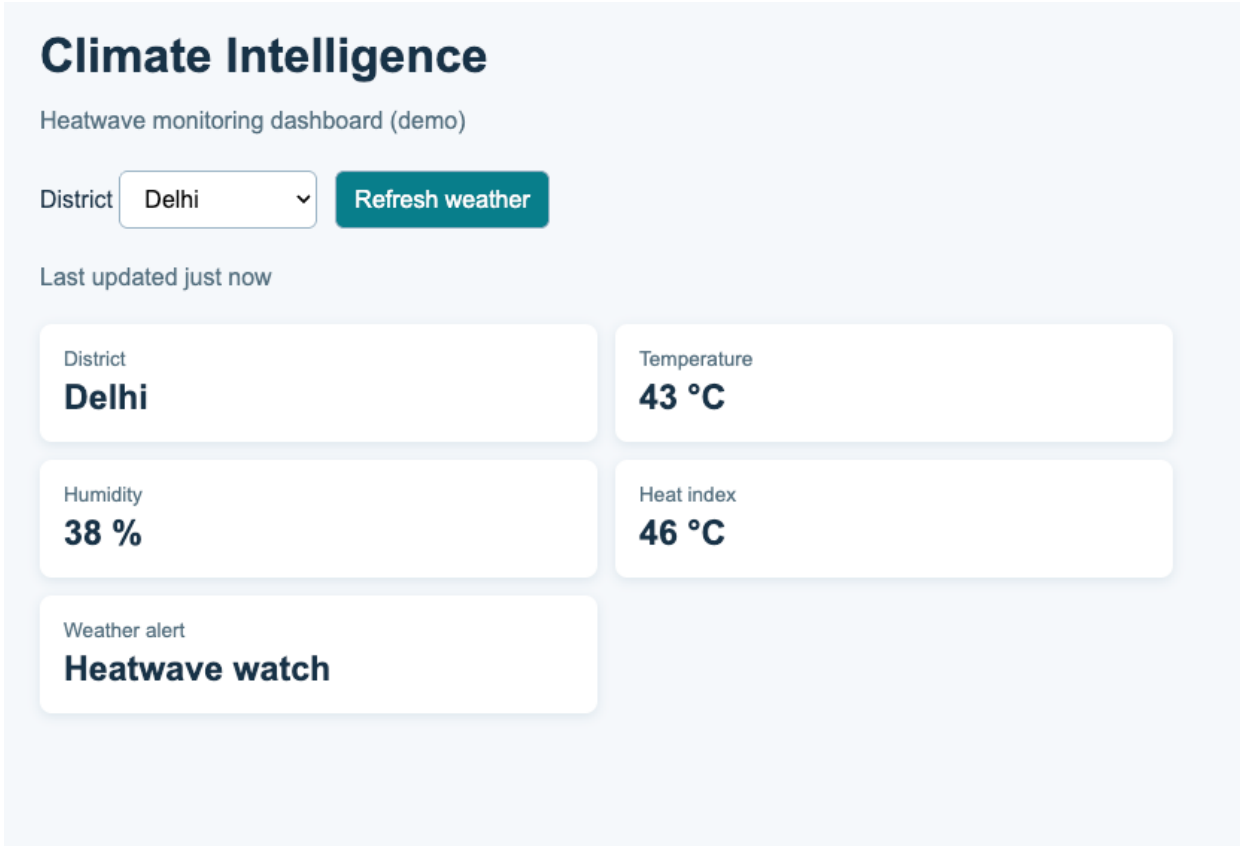

if __name__ == "__main__":
    main()
```

HTML:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Climate Intelligence Dashboard</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 760px; margin: 48px auto; padding: 0
20px; background: #f5f8fb; color: #183247; }
    h1 { margin-bottom: 6px; } .muted { color: #547080; }
    .controls { display: flex; gap: 12px; margin: 24px 0; }
    select, button { font: inherit; padding: 9px 12px; border-radius: 6px; border: 1px solid
#9bb4c5; }
    button { background: #087e8b; color: white; cursor: pointer; }
    #loading { min-height: 24px; color: #547080; }
    #weather-data { display: grid; grid-template-columns: repeat(2, 1fr); gap: 12px; }
    .card { background: white; border-radius: 8px; padding: 16px; box-shadow: 0 2px 8px
#dce6ed; }
    .label { display: block; font-size: .85rem; color: #547080; } .value { display: block;
font-size: 1.45rem; font-weight: bold; margin-top: 5px; }
  </style>
</head>
<body>
  <h1>Climate Intelligence</h1>
  <p class="muted">Heatwave monitoring dashboard (demo)</p>
  <div class="controls">
    <label>District <select id="district-
select"><option>Delhi</option><option>Jaipur</option><option>Ahmedabad</option></select></lab
el>
    <button id="refresh-button">Refresh weather</button>
  </div>
  <p id="loading">Select a district and refresh the dashboard.</p>
  <section id="weather-data" hidden>
    <div class="card"><span class="label">District</span><span id="district"
class="value"></span></div>
    <div class="card"><span class="label">Temperature</span><span id="temperature"
class="value"></span></div>
    <div class="card"><span class="label">Humidity</span><span id="humidity"
class="value"></span></div>
    <div class="card"><span class="label">Heat index</span><span id="heat-index"
class="value"></span></div>
```

```
    <div class="card"><span class="label">Weather alert</span><span id="weather-alert"
class="value"></span></div>
  </section>
  <script>
    const readings = {
      Delhi:     { temperature: 43, humidity: 38, heatIndex: 46, alert: 'Heatwave watch' },
      Jaipur:    { temperature: 46, humidity: 25, heatIndex: 47, alert: 'Severe heatwave
alert' },
      Ahmedabad: { temperature: 39, humidity: 65, heatIndex: 51, alert: 'High humidity
advisory' }
    };
    function refreshWeather() {
      const district = document.querySelector('#district-select').value;
      const data = readings[district];
      document.querySelector('#weather-data').hidden = true;
      document.querySelector('#loading').textContent = 'Loading live weather data...';
      // Simulates an API response from a dynamic weather dashboard.
      setTimeout(() => {
        document.querySelector('#district').textContent = district;
        document.querySelector('#temperature').textContent = `${data.temperature} °C`;
        document.querySelector('#humidity').textContent = `${data.humidity} %`;
        document.querySelector('#heat-index').textContent = `${data.heatIndex} °C`;
        document.querySelector('#weather-alert').textContent = data.alert;
        document.querySelector('#loading').textContent = 'Last updated just now';
        document.querySelector('#weather-data').hidden = false;
      }, 900);
    }
    document.querySelector('#refresh-button').addEventListener('click', refreshWeather);
  </script>
</body>
</html>
```

Output



```
(base) bhumit@bhumits-MacBook-Air climate-intelligence-selenium-demo %
python3 heatwave_scraper.py --district Jaipur --headless

EXTRACTED WEATHER DATA
district: Jaipur
temperature_c: 46.0
humidity_percent: 25.0
heat_index_c: 47.0
weather_alert: Severe heatwave alert

HEATWAVE PREDICTION
Risk level: EXTREME
Early warning: Issue an immediate public heatwave warning.
(base) bhumit@bhumits-MacBook-Air climate-intelligence-selenium-demo %
```

Post-Lab Theory Questions

1. What is the difference between By.CLASS_NAME and By.CSS_SELECTOR?
2. What is the difference between By.LINK_TEXT and By.PARTIAL_LINK_TEXT?
3. How is XPath used with Selenium?
4. What is the difference between driver.title and driver.current_url?
5. What is the purpose of click(), send_keys() and clear()??
6. Compare **Requests + BeautifulSoup** and **Selenium** for dynamic website scraping.

Answers:

1. By.CLASS_NAME is used to locate an element using its class name, while By.CSS_SELECTOR uses CSS selector syntax to locate elements based on classes, IDs, attributes, tags, and their combinations.
2. By.LINK_TEXT locates a link using its complete visible text, while By.PARTIAL_LINK_TEXT locates a link using only a part of its visible text.
3. XPath is used in Selenium to locate web elements based on their position, attributes, text, or relationships with other elements. It is useful when elements do not have simple IDs or class names.
4. driver.title returns the title of the current web page, while driver.current_url returns the complete URL of the current web page.
5. click() is used to click a web element, send_keys() is used to enter text or send keyboard inputs to an element, and clear() is used to remove existing text from input fields.
6. Requests + BeautifulSoup is simple and faster for scraping websites where the required data is present in the HTML source. Selenium controls a real browser and can handle JavaScript, dynamic content, buttons, forms, and user interactions, but it is generally slower and uses more resources.

Conclusion:

In conclusion, Selenium WebDriver is useful for scraping dynamic websites by locating elements, extracting data, and performing interactions. It handles JavaScript-generated content effectively and provides multiple locators and methods. Thus, Selenium is a powerful tool for browser automation and dynamic web data extraction.